SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)
Constituent of Symbiosis International (Deemed University), Pune
(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of india)
Re-Accredited by NAAC with 'A++' Grade
॥वसुधैव कुटुम्बकम्॥

CSE - Academic Year 2026-27
Java Programming Lab | Batch 2025-29

ASSIGNMENT : 30

Name of Student	Anushree Shinde
PRN No.	25070122041
Batch	CSE A - 2
Class	Java Programming Lab
Academic Year & Semester	2025-29 : Semester 3
Date of Submission	

TITLE : Write a JSP application to demonstrate CRUD operations.

Learning Objectives : To implement Create, Read, Update, and Delete operations using JSP and a database.

Theory : CRUD stands for Create, Read, Update, and Delete, which represent the four basic database operations performed in web applications. JSP works together with Servlets and JDBC to implement these operations. Users interact with HTML forms created using JSP, while JDBC executes SQL statements to manipulate database records. INSERT adds new records, SELECT retrieves information, UPDATE modifies existing data, and DELETE removes unwanted records. Proper validation, exception handling, and session management improve application reliability and security. JSP CRUD applications are widely used in employee management systems, online shopping portals, hospital management systems, banking applications, and other enterprise software.

Exercise :

1. JSP-based application to add, update, delete, and view employee details

```
<%@ page import="java.util.*" %>
<%
    Map<Integer, String[]> employees =
        (Map<Integer, String[]>) application.getAttribute("employees");

    if (employees == null) {
        employees = new LinkedHashMap<Integer, String[]>();
        application.setAttribute("employees", employees);
    }

    String action = request.getParameter("action");
    String message = "";

    if ("add".equals(action)) {
        int id = Integer.parseInt(request.getParameter("id"));
        String name = request.getParameter("name");
        String department = request.getParameter("department");
        double salary =
Double.parseDouble(request.getParameter("salary"));

        if (employees.containsKey(id)) {
            message = "Employee ID already exists!";
        } else {
            employees.put(id, new String[]{
                name, department, String.valueOf(salary)
            });
            message = "Employee added successfully!";
        }
    }

    if ("update".equals(action)) {
        int id = Integer.parseInt(request.getParameter("id"));

        if (employees.containsKey(id)) {
            employees.put(id, new String[]{
                request.getParameter("name"),
                request.getParameter("department"),
                request.getParameter("salary")
            });
            message = "Employee updated successfully!";
```

```
        } else {
            message = "Employee not found!";
        }
    }

    if ("delete".equals(action)) {
        int id = Integer.parseInt(request.getParameter("id"));

        if (employees.remove(id) != null) {
            message = "Employee deleted successfully!";
        } else {
            message = "Employee not found!";
        }
    }
%>

<!DOCTYPE html>
<html>
<head>
    <title>Employee Management System</title>
</head>
<body>

<h2>Employee Management System</h2>

<p><b><%= message %></b></p>

<h3>Add Employee</h3>

<form method="post">
    <input type="hidden" name="action" value="add">

    Employee ID:
    <input type="number" name="id" required><br><br>

    Name:
    <input type="text" name="name" required><br><br>

    Department:
    <input type="text" name="department" required><br><br>
```

```
    Salary:
    <input type="number" step="0.01" name="salary" min="0"
required><br><br>

    <input type="submit" value="Add Employee">
</form>

<hr>

<h3>Update Employee</h3>

<form method="post">
    <input type="hidden" name="action" value="update">

    Employee ID:
    <input type="number" name="id" required><br><br>

    New Name:
    <input type="text" name="name" required><br><br>

    New Department:
    <input type="text" name="department" required><br><br>

    New Salary:
    <input type="number" step="0.01" name="salary" min="0"
required><br><br>

    <input type="submit" value="Update Employee">
</form>

<hr>

<h3>Delete Employee</h3>

<form method="post">
    <input type="hidden" name="action" value="delete">

    Employee ID:
    <input type="number" name="id" required><br><br>

    <input type="submit" value="Delete Employee">
```

```
</form>

<hr>

<h3>Employee Records</h3>

<table border="1" cellpadding="8">
    <tr>
        <th>Employee ID</th>
        <th>Name</th>
        <th>Department</th>
        <th>Salary</th>
    </tr>

    <%
        for (Map.Entry<Integer, String[]> entry : employees.entrySet()) {
            String[] emp = entry.getValue();
    %>

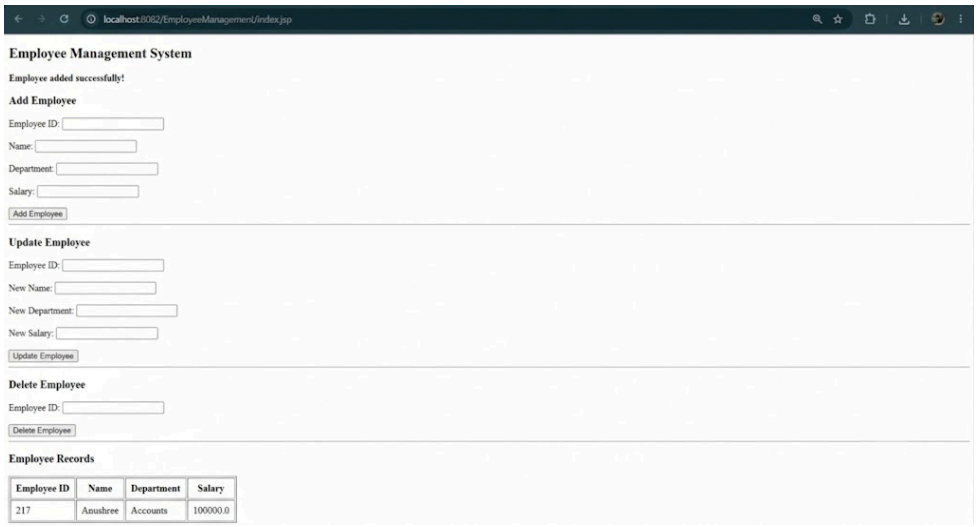
    <tr>
        <td><%= entry.getKey() %></td>
        <td><%= emp[0] %></td>
        <td><%= emp[1] %></td>
        <td><%= emp[2] %></td>
    </tr>

    <% } %>
</table>

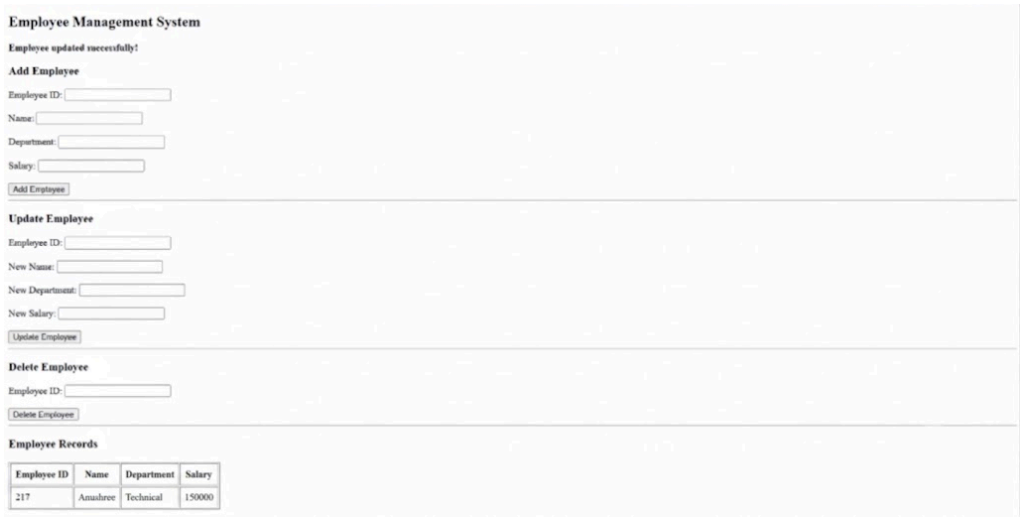
</body>
</html>
```

OUTPUT :

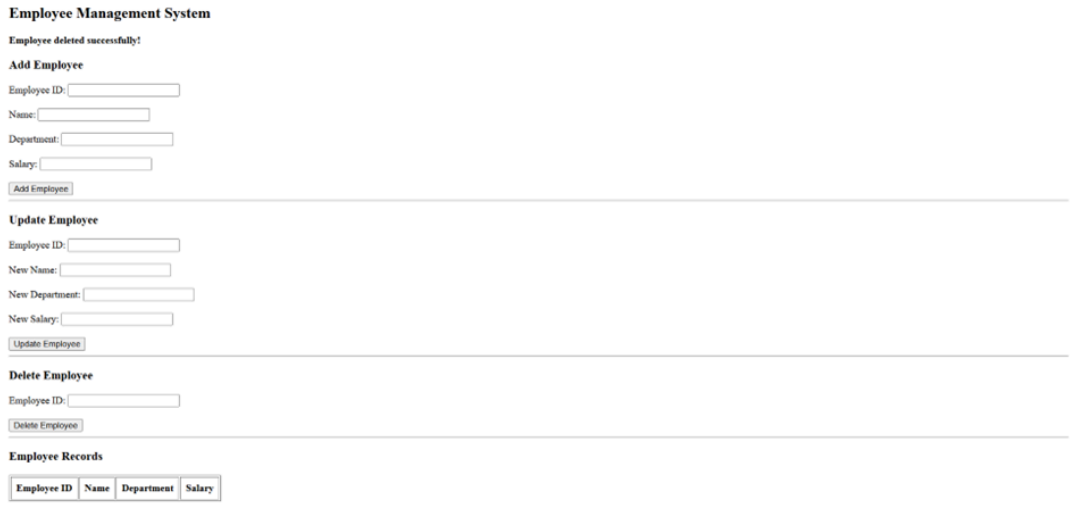
Insert :



Updating :



Deleting :



2. Create a JSP-based application to manage patient records using CRUD operations. Include fields such as patient ID, name, age, disease, and treatment status.

```
<%@ page import="java.sql.*" %>
<html>
<head>
    <title>Patient Management System</title>
</head>
<body>

<h2>Patient Management System</h2>

<%
String action = request.getParameter("action");

try {
    Class.forName("com.mysql.cj.jdbc.Driver");

    try (Connection con = DriverManager.getConnection(
        "jdbc:mysql://localhost:3306/studentdb", "root", "Sanika23@")) {

        if ("Add".equals(action)) {
            PreparedStatement ps = con.prepareStatement(
                "insert into patient values(?,?,?,?,?)");

            ps.setInt(1,
Integer.parseInt(request.getParameter("patient_id")));
            ps.setString(2, request.getParameter("name"));
            ps.setInt(3, Integer.parseInt(request.getParameter("age")));
            ps.setString(4, request.getParameter("disease"));
            ps.setString(5, request.getParameter("treatment_status"));

            ps.executeUpdate();
            ps.close();
            out.println("Patient added successfully!");
        }

        if ("Update".equals(action)) {
            PreparedStatement ps = con.prepareStatement(
```

```
                "update patient set name=?, age=?, disease=?,
treatment_status=? where patient_id=?");

            ps.setString(1, request.getParameter("name"));
            ps.setInt(2, Integer.parseInt(request.getParameter("age")));
            ps.setString(3, request.getParameter("disease"));
            ps.setString(4, request.getParameter("treatment_status"));
            ps.setInt(5,
Integer.parseInt(request.getParameter("patient_id")));

            int rows = ps.executeUpdate();
            ps.close();

            if (rows > 0)
                out.println("Patient updated successfully!");
            else
                out.println("Patient ID not found!");
        }

        if ("Delete".equals(action)) {
            PreparedStatement ps = con.prepareStatement(
                "delete from patient where patient_id=?");

            ps.setInt(1,
Integer.parseInt(request.getParameter("patient_id")));

            int rows = ps.executeUpdate();
            ps.close();

            if (rows > 0)
                out.println("Patient deleted successfully!");
            else
                out.println("Patient ID not found!");
        }
    }
} catch (Exception e) {
    out.println("Error: " + e.getMessage());
}
%>

<h3>Enter Patient Details</h3>
```

```
<form method="post">
    Patient ID:
    <input type="number" name="patient_id" required><br><br>

    Name:
    <input type="text" name="name" required><br><br>

    Age:
    <input type="number" name="age" min="0" required><br><br>

    Disease:
    <input type="text" name="disease" required><br><br>

    Treatment Status:
    <select name="treatment_status" required>
        <option value="Under Treatment">Under Treatment</option>
        <option value="Recovered">Recovered</option>
        <option value="Discharged">Discharged</option>
    </select><br><br>

    <button type="submit" name="action" value="Add">Add</button>
    <button type="submit" name="action" value="Update">Update</button>
    <button type="submit" name="action" value="Delete">Delete</button>
</form>

<h3>Patient Records</h3>

<table border="1" cellpadding="8">
<tr>
    <th>Patient ID</th>
    <th>Name</th>
    <th>Age</th>
    <th>Disease</th>
    <th>Treatment Status</th>
</tr>

<%
try {
    Class.forName("com.mysql.cj.jdbc.Driver");
```

```
    try (Connection con = DriverManager.getConnection(
        "jdbc:mysql://localhost:3306/studentdb", "root", "Sanika23@");
        Statement st = con.createStatement();
        ResultSet rs = st.executeQuery("select * from patient")) {

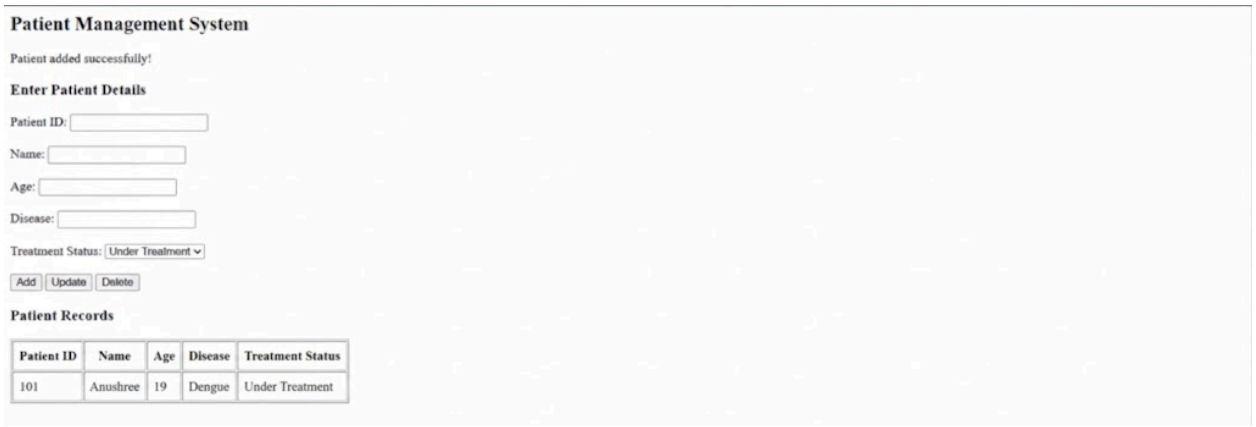
        while (rs.next()) {
%>
<tr>
    <td><%= rs.getInt("patient_id") %></td>
    <td><%= rs.getString("name") %></td>
    <td><%= rs.getInt("age") %></td>
    <td><%= rs.getString("disease") %></td>
    <td><%= rs.getString("treatment_status") %></td>
</tr>
<%
        }
    }
} catch (Exception e) {
    out.println("<tr><td colspan='5'>Error: " + e.getMessage() +
"</td></tr>");
}
%>

</table>

</body>
</html>
```

OUTPUT :

Insert :



Update :



Delete :



Outcome :

- Understood the concept of CRUD operations in database-driven web applications.
- Learned to implement Create, Read, Update, and Delete operations using JSP and JDBC.
- Understood the use of INSERT, SELECT, UPDATE, and DELETE SQL statements.
- Learned to use PreparedStatement, executeUpdate(), and ResultSet for database operations.
- Successfully implemented JSP-based applications for managing employee details and patient records.

Conclusion :

This assignment demonstrated the implementation of CRUD operations using JSP, JDBC, and a MySQL database. Employee and patient management applications were developed to add, retrieve, update, and delete database records. The assignment helped in understanding database connectivity, SQL query execution, and the management of records in dynamic web applications.